

Лабораторная работа №8

Отчет

Зубов Иван Александрович

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Выводы	10
4	Контрольные вопросы	11

Список иллюстраций

2.1	Устанавливаем Питон	6
2.2	Создаем файл	6
2.3	Готовый код	7
2.4	Определении функции	7
2.5	Перевод в байты и шифрование	8
2.6	Шестнадцатеричный вид	8
2.7	Восстановление текста	8
2.8	Смотрим результат	9

Список таблиц

1 Цель работы

Освоить на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом.

2 Выполнение лабораторной работы

Устанавливаем Python для дальнейшей работы с этим языком программирования

```
iazubov@iazubov:~$ sudo dnf install python3 -y
Extra Packages for Enterprise Linux 10 - x86_64                18 kB/s | 18 kB    00:00
Extra Packages for Enterprise Linux 10 - x86_64                2.8 MB/s | 6.6 MB  00:02
Rocky Linux 10 - BaseOS                                         9.8 kB/s | 4.3 kB  00:00
Rocky Linux 10 - BaseOS                                         1.8 MB/s | 1.7 MB  00:00
Rocky Linux 10 - AppStream                                       13 kB/s | 4.3 kB  00:00
Rocky Linux 10 - AppStream                                       2.3 MB/s | 1.7 MB  00:00
Rocky Linux 10 - Extras                                          8.6 kB/s | 3.1 kB  00:00
Rocky Linux 10 - Extras                                          4.7 kB/s | 5.8 kB  00:01
Пакет python3-3.12.12-3.el10_1.x86_64 уже установлен.
Зависимости разрешены.
=====
Пакет      Архитектура  Версия      Репозиторий  Размер
=====
Обновление:
  expat      x86_64       2.7.3-1.el10      baseos       120 k
  python-unversioned-command  noarch       3.12.13-2.el10_2  appstream    11 k
  python3     x86_64       3.12.13-2.el10_2  baseos       28 k
  python3-libs x86_64       3.12.13-2.el10_2  baseos       8.9 M
=====
Результат транзакции
=====
Обновление 4 Пакета

Объем загрузки: 9.1 M
Загрузка пакетов:
Rocky Linux 10 - BaseOS 196% [=====]
(1/4): expat-2.7.3-1.el10.x86_64.rpm 892 kB/s | 120 kB 00:00
(2/4): python3-3.12.13-2.el10_2.x86_64.rpm 198 kB/s | 28 kB 00:00
(3/4): python-unversioned-command-3.12.13-2.el10_2.noarch.rpm 215 kB/s | 11 kB 00:00
(4/4): python3-libs-3.12.13-2.el10_2.x86_64.rpm 9.4 MB/s | 8.9 MB 00:00
-----
Общий размер 5.5 MB/s | 9.1 MB 00:01
Проверка транзакции
Проверка транзакции успешно завершена.
Идет проверка транзакции
Тест транзакции проведен успешно.
Выполнение транзакции
Подготовка : 1/1
```

Рисунок 2.1: Устанавливаем Питон

Используем команду nano для создания и редактирования файла Python

```
ВЫПОЛНЕНО !
iazubov@iazubov:~$ nano lab8.py
```

Рисунок 2.2: Создаем файл

Так выглядит написанный код, в котором написано приложение для однократного гаммирования, который дальше я подробно поясню

```

GNU nano 8.1 lab8.py ИВМЕР
def xor_bytes(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

P1 = "НаВашскходящийот1204"
P2 = "ВСеверныйфилиалБанка"

K = bytes.fromhex(
    "05 0C 17 7F 0E 4E 37 D2 94 10 "
    "09 2E 22 57 FF C8 0B 82 70 54"
)

P1_bytes = P1.encode("cp1251")
P2_bytes = P2.encode("cp1251")

C1 = xor_bytes(P1_bytes, K)
C2 = xor_bytes(P2_bytes, K)

print("P1:", P1)
print("P2:", P2)
print("K :", K.hex(" ").upper())

print("\nШифротекст C1:")
print(C1.hex(" ").upper())

print("\nШифротекст C2:")
print(C2.hex(" ").upper())

print("\nC1 XOR C2:")
C1_xor_C2 = xor_bytes(C1, C2)
print(C1_xor_C2.hex(" ").upper())

print("\nВосстановление P2 без знания ключа:")
restored_P2 = xor_bytes(C1_xor_C2, P1_bytes)
print(restored_P2.decode("cp1251"))

print("\nВосстановление P1 без знания ключа:")
restored_P1 = xor_bytes(C1_xor_C2, P2_bytes)
print(restored_P1.decode("cp1251"))

```

Рисунок 2.3: Готовый код

В этой функции реализована операция XOR (исключающее ИЛИ) для двух последовательностей байтов. Функция поочередно берет байты из двух массивов и выполняет над ними XOR. Результат используется как при шифровании, так и при расшифровании.

Дальше задаются два открытых текста из условия лабораторной работы. Именно эти сообщения необходимо зашифровать одним и тем же ключом.

А также ключ шифрования в шестнадцатеричном формате. Метод `fromhex()` преобразует последовательность шестнадцатеричных чисел в массив байтов.

```

def xor_bytes(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

P1 = "НаВашскходящийот1204"
P2 = "ВСеверныйфилиалБанка"

K = bytes.fromhex(
    "05 0C 17 7F 0E 4E 37 D2 94 10 "
    "09 2E 22 57 FF C8 0B 82 70 54"
)

```

Рисунок 2.4: Определении функции

Операция XOR работает только с числами. Поэтому текст необходимо сначала

перевести в байтовое представление. Для русских символов используется кодировка CP1251.

И выполняем шифрование по формуле однократного гаммирования:

```
P1_bytes = P1.encode("cp1251")
P2_bytes = P2.encode("cp1251")

C1 = xor_bytes(P1_bytes, K)
C2 = xor_bytes(P2_bytes, K)
```

Рисунок 2.5: Перевод в байты и шифрование

Шифротексты выводятся в шестнадцатеричном виде для удобства анализа и сравнения результатов.

```
print("\nШифротекст C1:")
print(C1.hex(" ").upper())

print("\nШифротекст C2:")
print(C2.hex(" ").upper())

print("\nC1 XOR C2:")
C1_xor_C2 = xor_bytes(C1, C2)
print(C1_xor_C2.hex(" ").upper())
```

Рисунок 2.6: Шестнадцатеричный вид

Предположим, злоумышленник знает текст P1. Тогда он может найти второй текст без знания ключа. В программе выполняется именно эта операция: `restored_P2 = xor_bytes(C1_xor_C2, P1_bytes)`. Аналогично и с P2

```
print("\nВосстановление P2 без знания ключа:")
restored_P2 = xor_bytes(C1_xor_C2, P1_bytes)
print(restored_P2.decode("cp1251"))

print("\nВосстановление P1 без знания ключа:")
restored_P1 = xor_bytes(C1_xor_C2, P2_bytes)
print(restored_P1.decode("cp1251"))
```

Рисунок 2.7: Восстановление текста

Запуская наш файл и смотри результат.


```
lazubov@lazubov:~$ python3 lab8.py
P1: НВашисходящий1204
P2: ВСеверныйфилиалБанка
K : 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54

Шифротекст C1:
C8 EC D5 9F F6 A6 C6 27 7A F4 F6 D7 CA BE 11 3A 3A 80 40 60

Шифротекст C2:
C7 DD F2 9D EB BE DA 29 7D E4 E1 C5 CA B7 14 09 EB 5F 9A B4

C1 XOR C2:
0F 31 27 02 1D 18 1C 0E 07 10 17 12 00 09 05 33 D1 DF DA D4

Восстановление P2 без знания ключа:
ВСеверныйфилиалБанка
```

```
00: Восстановление P2 без знания ключа:
НВашисходящий1204
lazubov@lazubov:~$
```

Рисунок 2.8: Смотрим результат

3 Выводы

В ходе лабораторной работы был изучен режим однократного гаммирования и реализовано шифрование двух сообщений одним ключом. Были получены шифротексты С1 и С2 и показана уязвимость повторного использования ключа.

4 Контрольные вопросы

1. Как, зная один из текстов (P1 или P2), определить другой, не зная при этом ключа? Если известен один открытый текст, то второй можно найти по формуле: $P2 = C1 \oplus C2 \oplus P1$
2. Что будет при повторном использовании ключа при шифровании текста? Повторное использование ключа снижает безопасность и позволяет злоумышленнику получить информацию об открытых текстах.
3. Как реализуется режим шифрования однократного гаммирования одним ключом двух открытых текстов? Каждый открытый текст складывается по модулю 2 (XOR) с одним и тем же ключом: $C1 = P1 \oplus K$, $C2 = P2 \oplus K$.
4. Перечислите недостатки шифрования одним ключом двух открытых текстов. Снижается криптостойкость. Можно получить связь между сообщениями. При знании одного текста можно восстановить другой. Возможен криптоанализ без определения ключа.
5. Перечислите преимущества шифрования одним ключом двух открытых текстов. Простота реализации. Высокая скорость шифрования. Небольшие вычислительные затраты. Удобство использования одного ключа для шифрования данных.